

Classification of Classical Music Composers Using LSTM and CNN Architectures

Ahmed Eldahmy, Tony Diehl Verdugo, and Andrea Thomas

Department of Applied Artificial Intelligence, University of San Diego

AAI 511-03: Neural Networks and Deep Learning

Prof Bilgenur Baloglu

August 10, 2026

Author Note

Ahmed Eldahmy was responsible for dataset organization, preprocessing, feature extraction, visualizations, and saved model-ready outputs. Tony Diehl Verdugo was responsible for the LSTM model development, training, and evaluation. Andrea Thomas was responsible for the CNN model development, training, evaluation, final comparison, and report integration.

Abstract

This project explored the use of deep learning to classify classical music composers from MIDI files. The dataset included the four required composers: Bach, Beethoven, Chopin, and Mozart. The working dataset contained 1,630 MIDI files, and 1,628 files were successfully processed into feature records. Data augmentation via pitch shifting was implemented to expand the training data. The data was split at the file level into training, validation, and test sets before any LSTM windows were created. This was done to prevent data leakage between the splits. Two input formats were prepared from the same MIDI files: note and chord event sequences for the LSTM model and binary piano-roll matrices for the CNN model. The LSTM data used a training-only vocabulary with 39,934 tokens and a sliding window of 100 events. The CNN data used fixed 128 x 128 piano-roll matrices with one channel. These outputs were saved so the modeling team could train and compare the LSTM and CNN models using the same cleaned dataset. The processed dataset yielded 550,158 training windows for the LSTM and 2,276 training samples for the CNN.

Keywords: MIDI classification, composer identification, LSTM, CNN, preprocessing, music information retrieval

Classification of Classical Music Composers Using LSTM and CNN Architectures

In the data engineering and preprocessing portion of the final project, we focused on taking the raw MIDI files, cleaning and organizing them, and converting them into input formats that could be used by the LSTM and CNN models. This work was important because both models needed consistent labels, clean splits, and reusable processed files before training could begin.

Dataset Description

We sourced the dataset from the Kaggle MIDI Classic Music collection (Blanderbuss, 2019). The original collection includes classical MIDI files from many composers. Based on the project requirement, the working dataset was filtered to include only Bach, Beethoven, Chopin, and Mozart. We used one folder per composer and recursively searched each folder for files ending in .mid or .midi.

After filtering, the working dataset contained 1,630 MIDI files. Bach had the largest number of files, while Chopin had the fewest. This imbalance was important to document because a classifier can learn the majority class more easily if the imbalance is not considered during model evaluation. Table 1 shows the number of MIDI files used for each composer.

Table 1

MIDI File Count by Composer

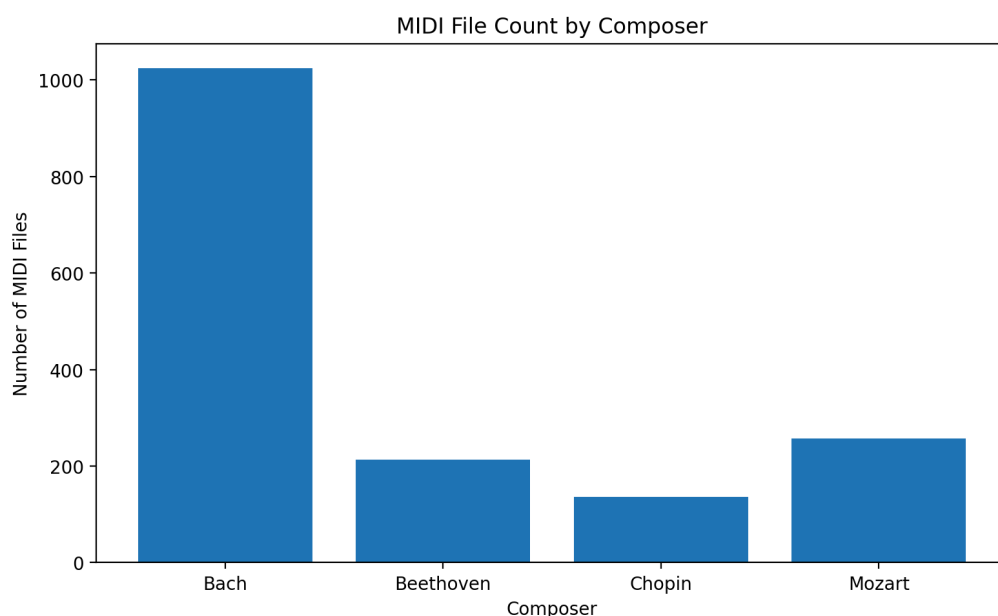
Composer	MIDI files
Bach	1,024
Beethoven	213
Chopin	136
Mozart	257
Total	1,630

Note. Counts are from the final Team Member 1 preprocessing notebook.

Figure 1 also shows the same distribution visually. The large number of Bach files and the smaller number of Chopin files were kept in mind when preparing the evaluation data for the team.

Figure 1

MIDI File Count by Composer



Note. The chart was generated in the preprocessing notebook before model training.

Data Preprocessing

We wrote the preprocessing pipeline in Python and used music21, pretty_midi, pandas, NumPy, scikit-learn, and Matplotlib. The main purpose was to create a clean and repeatable process that the other team members could use without redoing the raw MIDI parsing (Cuthbert & Ariza, 2010; Pedregosa et al., 2011; Raffel & Ellis, 2014).

File Validation and Cleaning

We collected every MIDI file path and assigned a composer label based on the folder name. Each file was then checked to make sure it could be parsed. All 1,630 files passed the first

validation step. During feature extraction, two files were skipped out of the original 1,140 training files. The final processed dataset therefore contained 1,140 files for training, 244 for validation, and 246 for testing.

Feature Extraction

We implemented data augmentation by duplicating training files and shifting their pitch by 2 semitones to create additional training features. This augmentation was explicitly restricted to the training split to prevent data leakage into the validation and test sets.

Train, Validation, and Test Split

We split the data using a 70%, 15%, and 15% structure for training, validation, and testing. The split was done at the file level and was stratified by composer. This means each composer was represented in each split while keeping complete MIDI files in only one split.

Splitting at the file level was important because the LSTM inputs were later created with overlapping sliding windows. If the windows had been created before the split, windows from the same MIDI file could have appeared in both the training and test data. That would create data leakage and make the model results look stronger than they really are (Kaufman et al., 2012). By splitting files first, we avoided this issue.

Feature Extraction

Two feature representations were created from the same MIDI files. The LSTM model needed ordered musical sequences, while the CNN model needed image-like piano-roll matrices. Creating both formats from the same file splits allowed the team to compare the models fairly.

Note and Chord Sequences for LSTM

For the LSTM model, we parsed each MIDI file with music21 and extracted notes and chords in order. Individual notes were stored as pitch tokens, such as C4 or G#5. Chords were

stored as dot-separated MIDI pitch values, such as 60.64.67, after sorting the chord pitches.

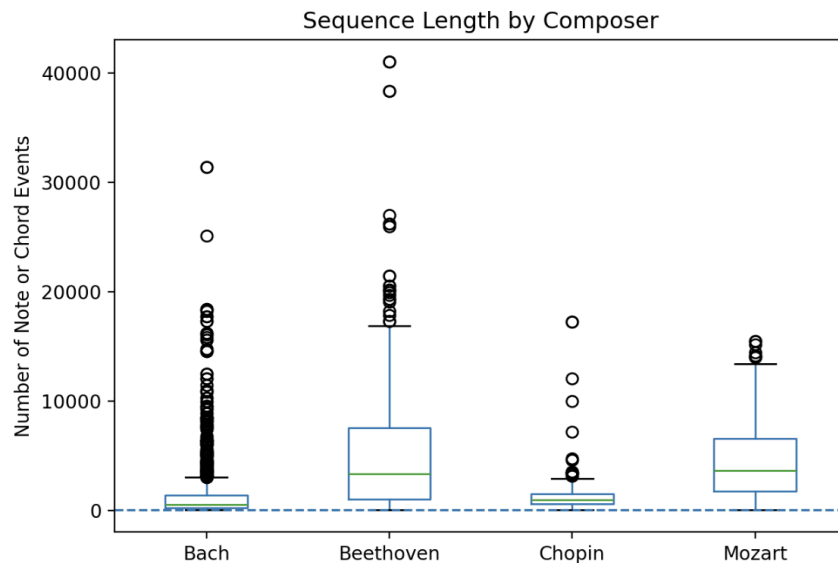
Sorting the chord pitches helped make the chord token consistent even if the notes appeared in a different internal order inside the MIDI file.

We also extracted tempo when it was available. When no tempo marking was found, the default value was set to 120 beats per minute. Other summary values were collected for reporting, including sequence length, total duration in quarter lengths, mean pitch, and normalized mean pitch.

The LSTM vocabulary was built from the training set only. This produced 39,934 unique tokens. Using only the training set for the vocabulary avoided information leakage from the validation and test sets. Unknown tokens were mapped to the reserved index 0. We then created fixed-length windows of 100 events with a step size of 10. This produced 550,158 training windows, 65,888 validation windows, and 60,833 test windows. Sequence length varied significantly by composer, especially for Beethoven and Mozart. Figure 2 shows this variation and helps explain why fixed-length LSTM windows were needed.

Figure 2

Sequence Length by Composer



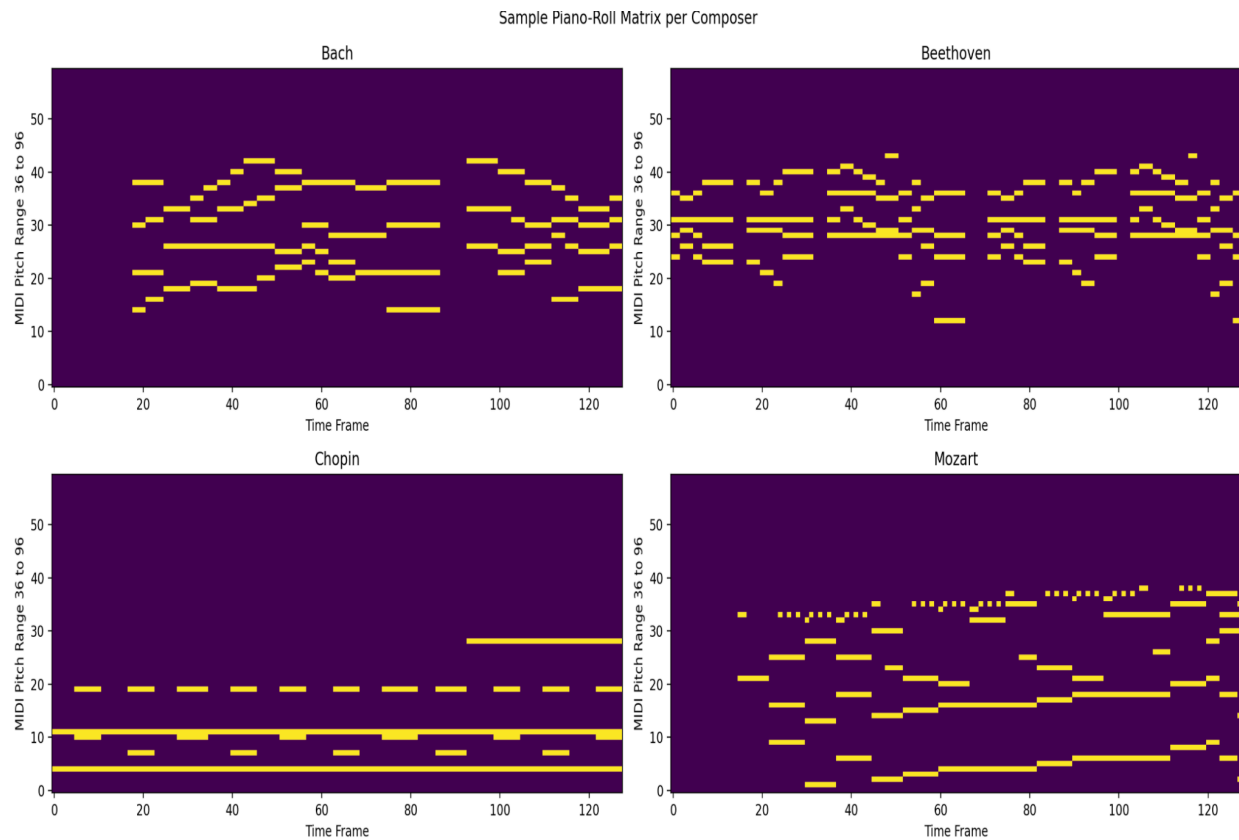
Note. Sequence length is measured by the number of note or chord events extracted from each usable MIDI file.

Piano-Roll Matrices for CNN

For the CNN model, each MIDI file was converted into a binary piano-roll matrix using `pretty_midi`. A piano roll represents music as a grid where pitch is on one axis and time is on the other. We represented each piano roll using 128 MIDI pitch bins and sampled the audio at 10 frames per second. A value of 1 means a pitch was active at that frame, and a value of 0 means it was not active.

To keep the CNN input consistent, every piano roll was converted to a fixed 128 x 128 shape. Longer rolls were truncated to 128 frames, and shorter rolls were padded with zeros. A channel dimension was then added, making the final CNN input shape (128, 128, 1). Each usable MIDI file produced one CNN matrix. The final CNN arrays contained 2,276 training samples, 244 validation samples, and 246 test samples.

Figure 3 shows examples of piano-roll matrices from the preprocessing notebook. This helped confirm that the CNN input was being generated correctly before the model training step.

Figure 3*Sample Piano-Roll Matrices*

Note. The visualizations show binary pitch activity over time for sample MIDI files from the four composers.

Prepared Outputs for the Team

After preprocessing and feature extraction, the notebook saved the final arrays and metadata files in the `processed_data` folder. These files were shared with the team so the LSTM and CNN notebooks could load the processed data directly. This made the project easier to reproduce and helped keep the modeling sections consistent.

Table 2*Saved Model-Ready Outputs*

Output group	Files	Purpose
LSTM arrays	X_seq_train, X_seq_val, X_seq_test; y_seq_train, y_seq_val, y_seq_test	Sequence inputs and labels for LSTM training and evaluation
CNN arrays	X_cnn_train, X_cnn_val, X_cnn_test; y_cnn_train, y_cnn_val, y_cnn_test	Piano-roll inputs and labels for CNN training and evaluation
Metadata	label_classes.npy, token_to_int.json, dataset_summary.json	Label mapping, vocabulary, and preprocessing settings
Reports and splits	feature_summary.csv, file_level_splits.csv	Feature-level summary and file-level split tracking
Figures	composer_distribution.png, sequence_length_by_composer.png, note_frequency_by_composer.png, sample_piano_rolls.png	Visual checks and report figures

Note. File names are shown without the .npy extension when grouped for readability.

Data Pipeline Summary and Future Improvements

The preprocessing section produced a clean and reusable dataset pipeline for the full project. The same MIDI files were turned into two model-ready formats: note and chord sequences for the LSTM model and piano-roll matrices for the CNN model. The notebook also saved the label encoding, vocabulary, split file, feature summary, and visualizations so the team could reproduce the work later.

A key finding from the data preparation stage was that the dataset was not balanced across composers. Bach had far more files than the other composers, while Chopin had the

fewest. This should be considered when interpreting model results. Accuracy alone may not be enough, so the final model evaluation should also review precision, recall, F1-score, and the confusion matrix.

Future improvements could include adding more Chopin, Beethoven, and Mozart MIDI files to reduce class imbalance, testing longer piano-roll windows for the CNN, and trying additional musical features such as note density, rhythm patterns, and key signatures.

LSTM Model Development

LSTM Methodology

We built a Long Short-Term Memory (LSTM) model to classify symbolic musical sequences into one of four composer classes: Bach, Beethoven, Chopin, and Mozart. Unlike the CNN model, which uses piano-roll images, the LSTM model processes sequential note and chord events extracted from the MIDI files. Each musical event was converted into an integer representation using the vocabulary generated during preprocessing. Unknown musical events were assigned to a reserved token to maintain consistency across the validation and test datasets.

The processed input sequences consisted of fixed-length windows of 100 musical events. These integer-encoded sequences were loaded directly from the preprocessing outputs created by Team Member 1. Because the dataset contained an uneven distribution of composers, balanced class weights were calculated from the training labels and incorporated during model training to reduce bias toward the majority class. The resulting training vocabulary contained 39,934 unique musical tokens, which formed the input vocabulary for the embedding layer.

LSTM Architecture

The baseline LSTM architecture consisted of an embedding layer with 128 dimensions, followed by a single LSTM layer containing 128 hidden units. A dropout layer with a rate of

0.30 was applied after the LSTM layer to reduce overfitting. A fully connected dense layer with 64 neurons and ReLU activation further transformed the learned sequence representation before the final softmax output layer predicted one of the four composer classes.

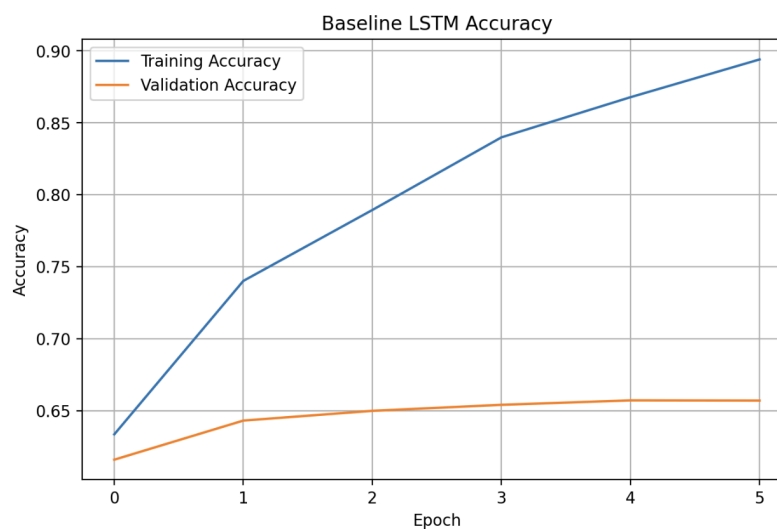
We compiled using the Adam optimizer with an initial learning rate of 0.001 and sparse categorical cross-entropy loss. Sparse categorical cross-entropy was selected because the composer labels were stored as integer values rather than one-hot encoded vectors.

Training Procedure

We trained the baseline LSTM model using a batch size of 128 for a maximum of 15 epochs. Three callback functions were incorporated to improve training performance and reduce overfitting. Early stopping monitored validation loss and restored the best-performing model weights once validation performance stopped improving. Model checkpointing automatically saved the best model during training, while ReduceLROnPlateau decreased the learning rate whenever validation loss plateaued. Training and validation accuracy and loss were monitored throughout the training process to evaluate convergence and identify potential overfitting.

Figure 4

Baseline LSTM Training and Validation Accuracy



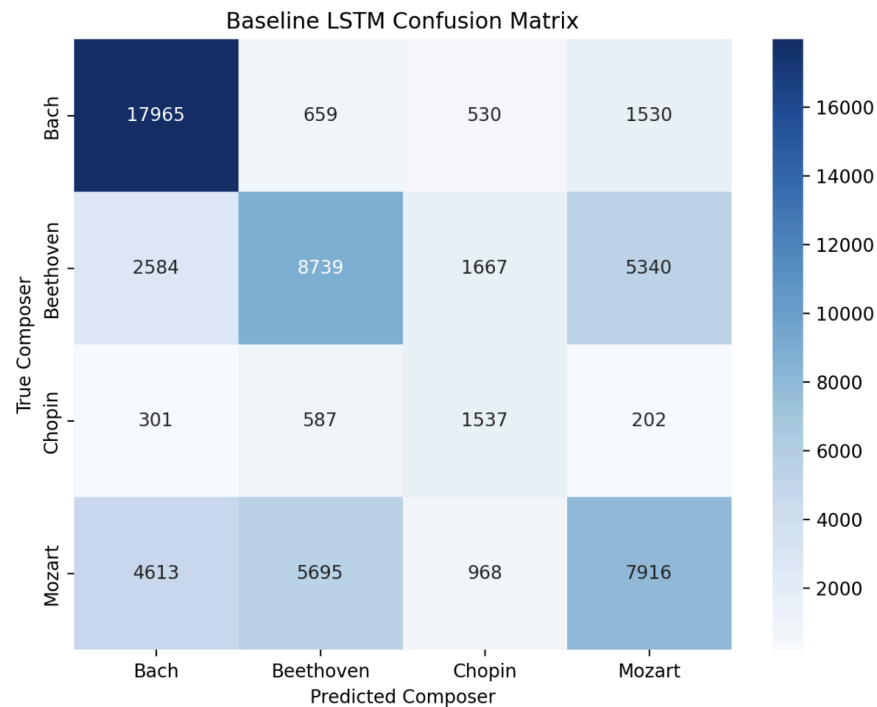
Note. Training and validation accuracy for the baseline LSTM model over 5 training epochs.

Baseline LSTM Results

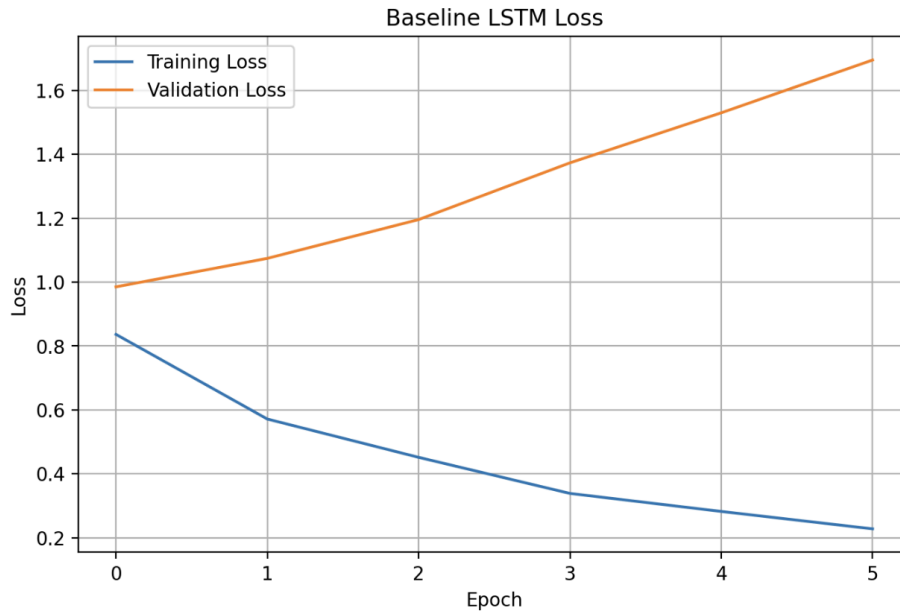
The baseline LSTM successfully learned meaningful sequential patterns from the encoded musical data. Final training accuracy reached 89.41%, while validation accuracy reached 65.71%. Evaluation on the unseen test dataset produced a test accuracy of 59.44% with a test loss of 1.0175. The baseline model exhibited significant overfitting, with validation loss climbing to 1.6948 by the final epoch. Early stopping restored the model weights from the epoch that achieved the lowest validation loss, helping reduce additional overfitting during training. Figure 5 illustrates the baseline confusion matrix and shows that Bach was classified more accurately than the remaining composers.

Figure 5

Baseline LSTM Confusion Matrix



Note. Confusion matrix showing the baseline LSTM composer classifications on the test dataset.

Figure 6*Baseline LSTM Training and Validation Loss*

Note. Training and validation loss for the baseline LSTM during model training.

Improved LSTM Model

To investigate whether additional model complexity could improve performance, we developed a second LSTM architecture. The embedding dimension was increased from 128 to 256, and a second LSTM layer containing 64 hidden units was added after the first LSTM layer. Dropout regularization was increased from 0.30 to 0.40 throughout the network, and the dense hidden layer was expanded from 64 to 128 neurons. The learning rate was reduced to 0.0005 to promote more stable optimization during training. We trained the improved model using the same preprocessing pipeline, batch size, class weights, callbacks, and evaluation procedures as the baseline model to ensure a fair comparison.

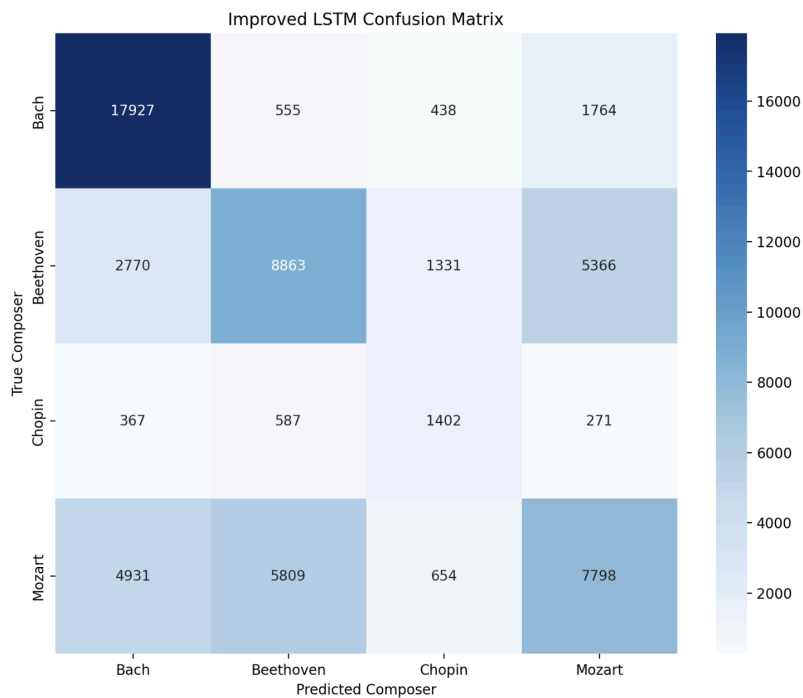
Improved LSTM Results

The improved architecture achieved a higher final training accuracy of 85.10% and a slightly higher validation accuracy of 66.39%. However, evaluation on the unseen test dataset produced a lower test accuracy of 59.16%, despite achieving a slightly lower test loss of 0.9969.

These results indicate that the additional model complexity increased the model's ability to fit the training data but did not improve its ability to generalize to unseen musical sequences. Validation loss increased after the first epoch despite continued improvements in training accuracy, demonstrating stronger overfitting than the baseline architecture. Early stopping restored the best-performing model weights before evaluation on the test dataset. Figure 7 shows that the improved architecture produced similar classification patterns despite its greater model complexity.

Figure 7

Improved LSTM Confusion Matrix



Note. Confusion matrix for the improved LSTM model evaluated on the test dataset.

LSTM Model Discussion

We compared the baseline and improved LSTM architectures to evaluate the effect of increasing model complexity. While the improved architecture achieved higher training accuracy and slightly higher validation accuracy, these improvements did not translate into improved performance on the independent test dataset. The baseline model achieved the higher test accuracy (59.44%) and demonstrated better overall generalization than the improved architecture.

The baseline model actually exhibited more severe overfitting (higher validation loss) than the improved model, even though the baseline still generalized slightly better on the final test accuracy. These findings illustrate an important principle in deep learning: increasing network depth and model capacity does not necessarily improve predictive performance. Additional complexity can increase the likelihood of overfitting when the available training data is limited or imbalanced. Based on the evaluation metrics, the baseline LSTM was selected as the stronger sequence model for this project.

Table 3

Comparison of Baseline and Improved LSTM Models

Metric	Baseline LSTM	Improved LSTM
Final Training Accuracy	89.41%	85.10%
Highest Validation Accuracy	65.71%	66.39%
Lowest Validation Loss	1.6948	1.0473
Test Accuracy	59.44%	59.16%
Test Loss	1.0175	0.9969
Overfitting	High	Moderate

CNN Model Development

CNN Methodology

In the final part of the project, we framed composer classification as an image-based pattern recognition task by representing symbolic MIDI music as binary piano-roll matrices. A piano roll transforms temporal musical structures into a two-dimensional grid where pitch occupies one axis and time frames occupy the other (Raffel & Ellis, 2014). We used 128 MIDI pitch bins and sampled time at 10 frames per second over a fixed window of 128 time frames, corresponding to 12.8 seconds of continuous musical audio. Active notes received a value of 1, whereas inactive frames received a value of 0. We truncated longer MIDI scores to 128 frames and padded shorter pieces with zeros.

We added a single channel dimension to produce an input tensor shape for each MIDI file. Because convolutional neural networks excel at detecting local spatial motifs across frequency and time axes, this representation allows the model to learn harmonic structures, polyphonic textures, and melodic contours directly from two-dimensional representations. We preserved the strict train, validation, and test file splits established during preprocessing to prevent data leakage (Kaufman et al., 2012). Each usable MIDI file produced exactly one piano-roll matrix.

CNN Architecture

We constructed a baseline Convolutional Neural Network using the Sequential API in TensorFlow and Keras. The network consists of three sequential convolutional blocks followed by a fully connected classifier head. The first block applies a two-dimensional convolutional layer with 32 filters, a 3x3 kernel size, and ReLU activation. A max pooling layer with a 2x2 pool size reduces spatial dimensions by half. The second block applies a convolutional layer with

64 filters and a 3x3 kernel size with ReLU activation, followed by another 2x2 max pooling layer. We flattened the multi-dimensional feature maps into a one-dimensional vector following the final pooling layer. A dense hidden layer containing 128 units with ReLU activation processed the extracted features. We inserted a dropout layer with a rate of 0.50 after the dense layer to reduce overfitting (Srivastava et al., 2014). The final output layer used a dense layer with four units and a softmax activation function to produce probability distributions across the composer classes.

To classify the extracted feature maps into composer categories, we flattened the multidimensional spatial tensor into a one-dimensional vector. We connected this vector to a dense layer containing 128 hidden units with ReLU activation and inserted a dropout layer with a rate of 0.50 after the dense hidden layer to reduce overfitting. The final dense output layer contains four neurons with softmax activation, yielding probability distributions across the target composer classes for Bach, Beethoven, Chopin, and Mozart.

CNN Training

We compiled the baseline CNN using the Adam optimizer with default hyperparameter settings and sparse categorical cross-entropy loss (Kingma & Ba, 2014). We selected sparse categorical cross-entropy loss because the target composer labels were formatted as integer indices rather than one-hot vectors.

We incorporated two callbacks to streamline training and prevent overfitting and configured early stopping to monitor validation loss with a patience of five epochs, automatically restoring the best model weights once validation loss stopped improving. We also used model checkpointing to save the best model weights directly to disk. We then tracked total training

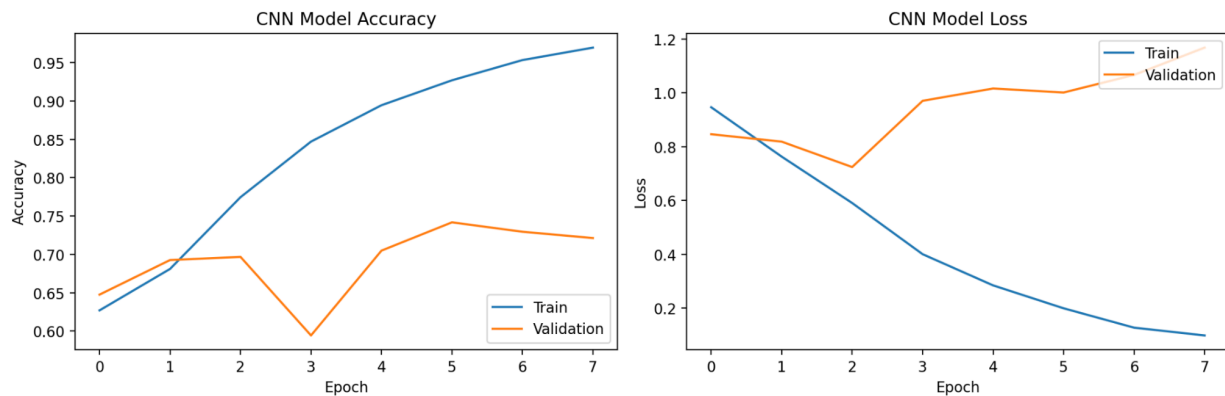
execution time to compare computational efficiency between two-dimensional convolutional processing and sequential LSTM processing.

Baseline CNN Results

We evaluated the saved baseline CNN model on the unseen test dataset to assess generalization performance. To obtain a complete evaluation, we calculated overall test loss and test accuracy alongside class-wise precision, recall, and weighted average F1-scores. We generated a confusion matrix to examine misclassification patterns across individual composers. Monitoring the learning curves helped us evaluate convergence behavior and identify the onset of overfitting, where training performance continues to improve while validation loss begins to rise. Restoring weights from the epoch with the lowest validation loss ensured that our test set predictions reflected optimal model parameters.

Figure 8

CNN Model Accuracy and Loss



Note. The CNN model shows signs of overfitting after the third epoch, where training accuracy and loss continue to improve, while validation performance begins to diverge and fluctuate. This indicates the model has begun to memorize the training data rather than generalizing to unseen patterns.

Improving the CNN Model

We developed an improved CNN architecture to enhance generalization, reduce overfitting, and stabilize network optimization. We introduced batch normalization layers after each convolutional layer and dense layer to normalize internal activation distributions and speed up convergence (Ioffe & Szegedy, 2015). We applied L2 weight regularization with a decay parameter of 0.001 across all convolutional and dense layers to penalize extreme network weights and restrict model complexity (Goodfellow et al., 2016).

We updated the convolutional layers to use the same padding, preserving spatial dimensions prior to downsampling. We added progressive dropout regularization inside each convolutional block, placing dropout rates of 0.25 after the first two max-pooling layers and 0.30 after the third max-pooling layer. The dense layer retained a 0.50 dropout rate alongside batch normalization and L2 regularization.

We compiled the improved architecture using the Adam optimizer with a reduced initial learning rate of 0.0005 to ensure stable weight updates. We modified the callback setup by increasing early stopping patience and adding a learning rate reduction callback. The learning rate callback tracked validation loss and halved the learning rate whenever validation loss plateaued across consecutive training epochs.

Table 4

Comparison of Baseline and Improved CNN Models

Metric	Baseline CNN	Improved CNN
Test Accuracy (%)	70.73%	75.61%
Test Loss	0.715	0.948
Precision	0.677	0.740
Recall	0.707	0.756

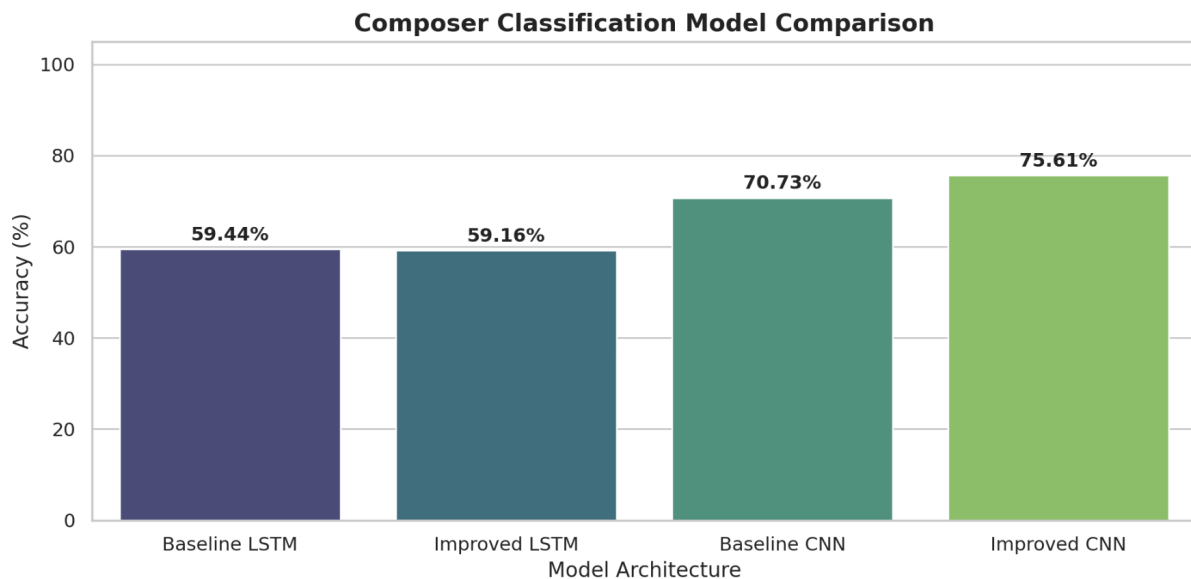
F1-Score	0.688	0.741
----------	-------	-------

Model Comparison

We evaluated both the baseline and improved CNN architectures against the LSTM ones to determine the best approach for composer classification. The CNN models, which processed musical data as two-dimensional piano-roll matrices, consistently produced higher generalization metrics than the sequence-based LSTM approaches. We found the improved CNN was the top-performing model overall, achieving a test accuracy of 75.6% and a weighted F1-score of 0.741. Our baseline CNN also performed strongly, recording a test accuracy of 70.7% and the lowest overall test loss of 0.715.

Figure 9

CNN and LSTM Model Comparison



Note. Comparison of final test set accuracy across all four models, demonstrating superior performance by the CNN architectures.

Conversely, the LSTM architectures, which relied on sequential note and chord events, yielded lower performance metrics and exhibited greater overfitting. We recorded a test accuracy of 59.4% and a test loss of 1.018 for the baseline LSTM. When we introduced additional complexity in the improved LSTM, the test accuracy slightly decreased to 59.2%, indicating that the increased capacity did not improve the model's ability to generalize to unseen sequences. Ultimately, we concluded that framing the classification as an image-based pattern recognition task was more effective for this dataset. The spatial hierarchies learned by the CNNs captured the harmonic structures and polyphonic textures of the classical composers more successfully than the temporal, sequence-based LSTM approach, providing a superior balance of precision, recall, and robust generalization.

Conclusion

In summary, our analyses demonstrate that increasing model complexity through deeper architectures can enhance learning capacity, but it does not always guarantee better generalization. For the LSTM architecture, our baseline model achieved a test accuracy of 59.4%, while the improved architecture achieved 59.2%. For the CNN architecture, the baseline model achieved a test accuracy of 70.7%, while the improved CNN model achieved 75.6%. These results highlight the importance of balancing model capacity with dataset size to mitigate overfitting.

References

- Blanderbuss. (2019). *MIDI classic music* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/blanderbuss/midi-classic-music>
- Cuthbert, M. S., & Ariza, C. (2010). music21: A toolkit for computer-aided musicology and symbolic music data. In *Proceedings of the 11th International Society for Music Information Retrieval Conference* (pp. 637-642). <https://zenodo.org/records/1416114>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning* (pp. 448-456). PMLR. <https://proceedings.mlr.press/v37/ioffe15.html>
- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 1-21. <https://doi.org/10.1145/2382577.2382579>
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*. <https://doi.org/10.48550/arXiv.1412.6980>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
<https://www.jmlr.org/papers/v12/pedregosa11a.html>
- Raffel, C., & Ellis, D. P. W. (2014). Intuitive analysis, creation and manipulation of MIDI data with pretty_midi. *Proceedings of the 15th International Society for Music Information*

Retrieval Conference Late Breaking and Demo Papers.

<https://colinraffel.com/publications/ismir2014intuitive.pdf>

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout:

A simple way to prevent neural networks from overfitting. *Journal of Machine Learning*

Research, 15(1), 1929-1958. <http://jmlr.org/papers/v15/srivastava14a.html>